

基于BP神经网络的自整定PID控制 仿真实验

人工智能系 晁飞

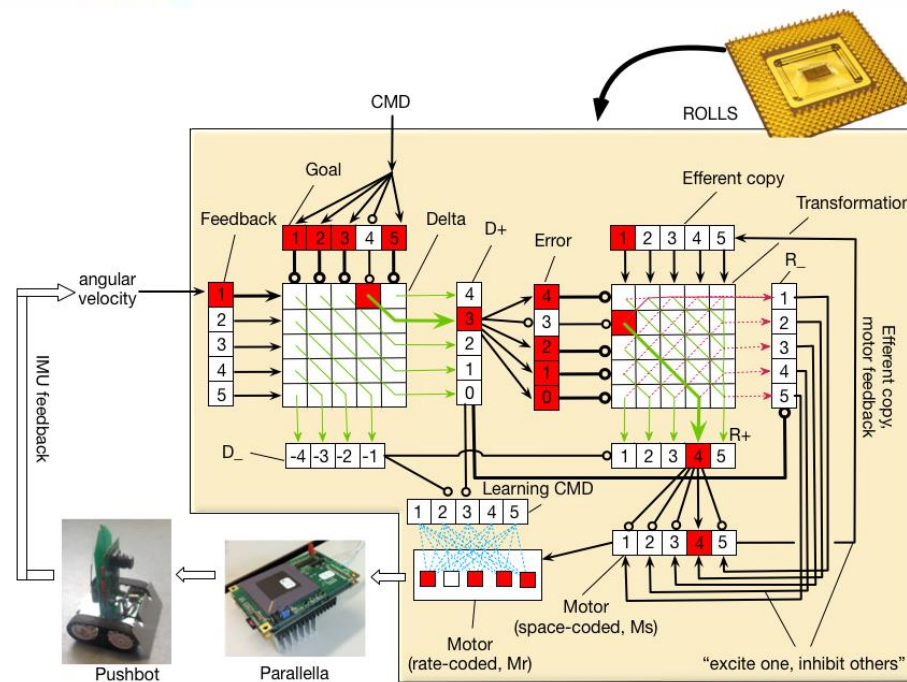
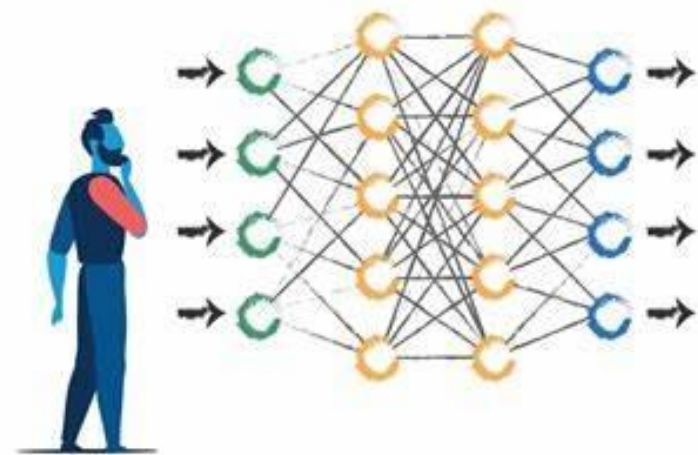
fchao@xmu.edu.cn

2023年7月19日



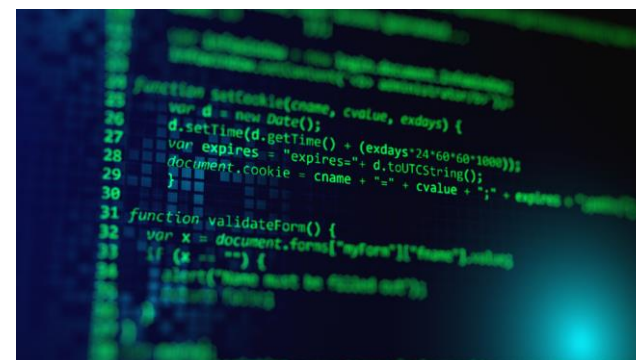
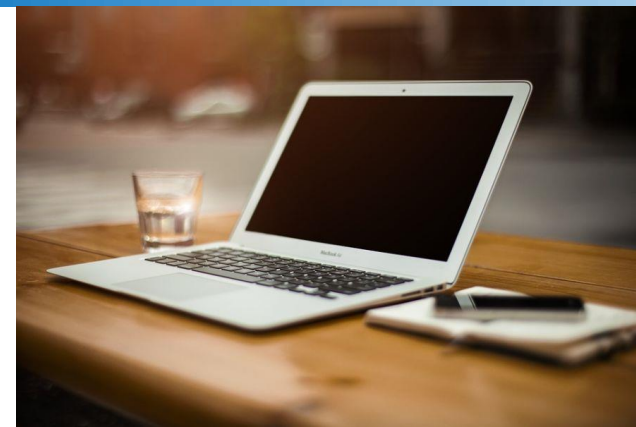
一，实验目的

1. 熟悉神经网络的特征、结构及学习算法。
2. 通过实验掌握神经网络自整定PID 的工作原理。
3. 了解神经网络的结构对控制效果的影响。
4. 掌握用Matlab 实现神经网络控制系统仿真的方法。



二，实验设备

- PC计算机设备
- Matlab仿真软件
- 实验程序代码

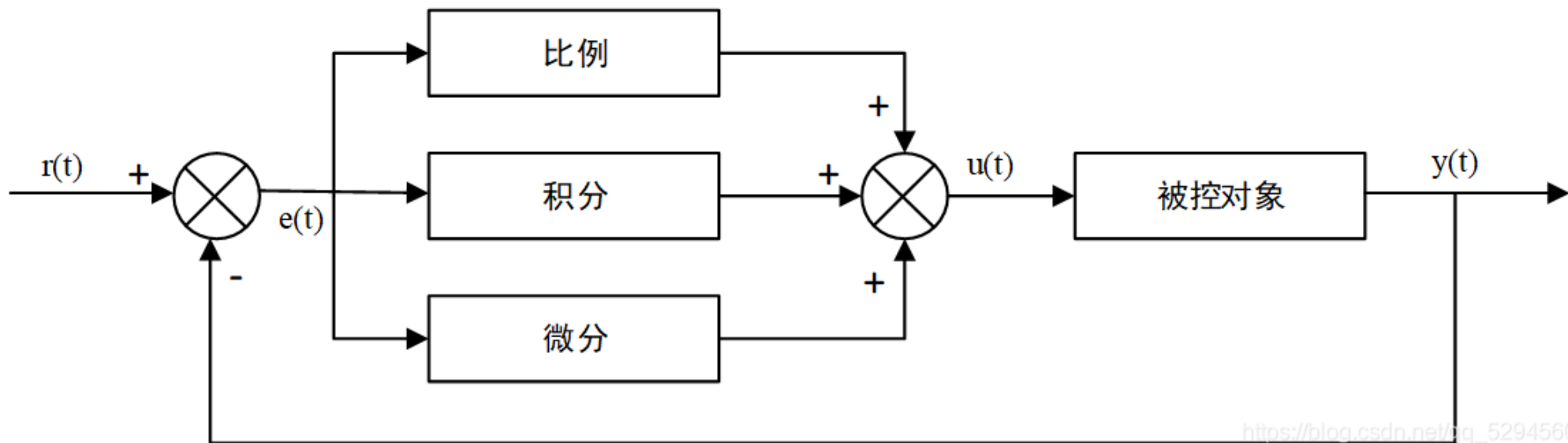


二，实验背景

在工业控制中，PID控制是工业控制中最常用的方法。这是因为PID控制器结构简单、实现简单，控制效果良好，已得到广泛应用。

但是，PID具有一定的局限性：被控制对象参数随时间变化时，控制器的参数难以自动调整以适应外界环境的变化。

为了使控制器具有较好的自适应性，实现控制器参数的自动调整，可以采用神经网络控制的方法。利用人工神经网络的自学习这一特性，并结合传统的PID控制理论，构造神经网络PID控制器，实现控制器参数的自动调整。

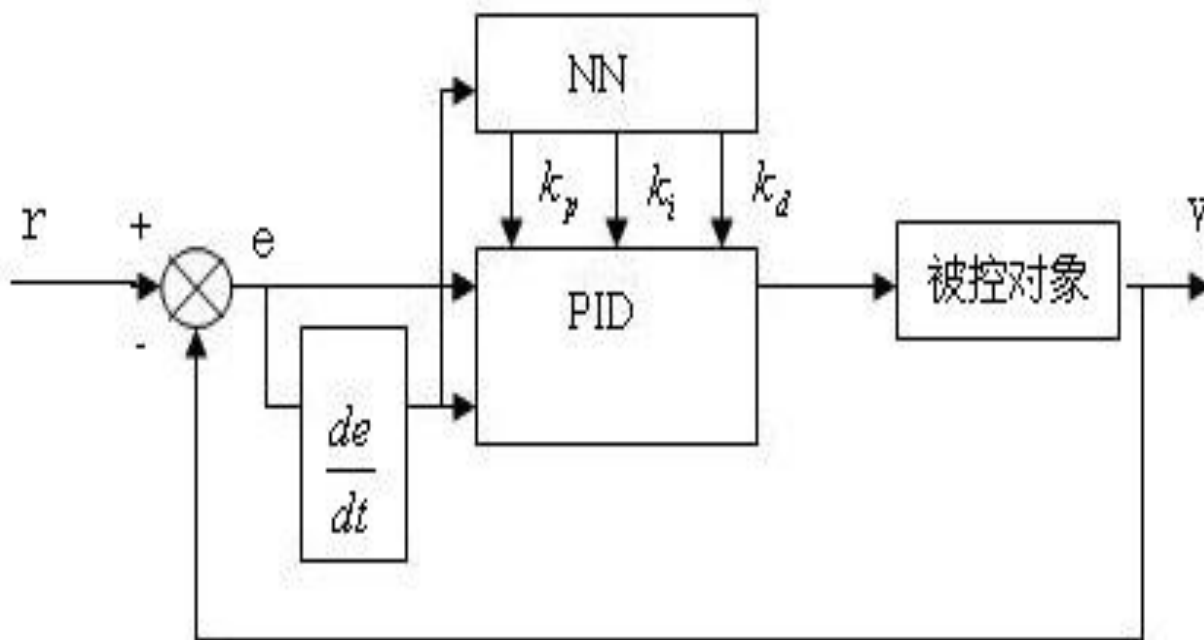


三，实验原理

基于BP神经网络的PID控制器结构如图所示。控制器由两部分组成：

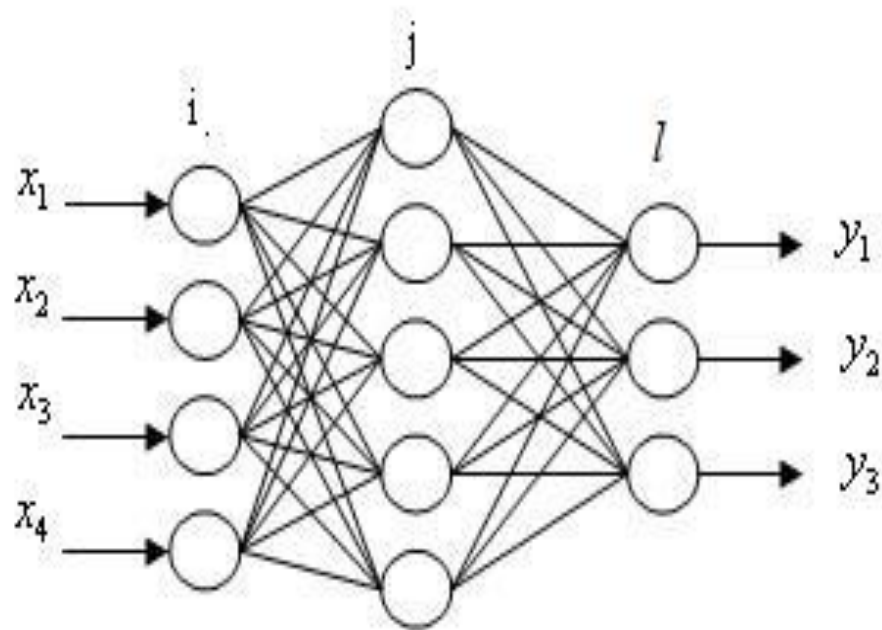
一是常规PID控制器，用以直接对对象进行闭环控制，且三个参数在线整定

二是神经网络NN，根据系统的运行状态，学习调整权系数，从而调整PID参数，达到某种性能指标的最优化



基于BP神经网络的PID控制器结构如图所示。控制器由两部分组成：

- 一是常规PID控制器，用以直接对对象进行闭环控制，且三个参数在线整定
- 二是神经网络NN，根据系统的运行状态，学习调整权系数，从而调整PID参数，达到某种性能指标的最优化



三，实验原理



被控对象为一时变非线性对象:

系数 $a(k)$ 是慢时变的

为保证控制器有一定的动态跟踪能力，选定神经网络的输入层输入为

输出层节点分别对应三个可调参数

$$y(k) = \frac{a(k)y(k-1)}{1+y^2(k-1)} + u(k-1)$$

$$a(k) = 1.2(1 - 0.8e^{-0.1k})$$

$$X_{in} = [e(k), e(k-1), e(k-2), 1]^T$$

$$\left. \begin{aligned} O_1^{(3)} &= K_p \\ O_2^{(3)} &= K_I \\ O_3^{(3)} &= K_D \end{aligned} \right\}$$

三，实验原理



取性能指标函数为：

$$E(k) = \frac{1}{2} (r(k) - y(k))^2$$

每一次的 $e(k)$ 为

$$r(k) - y(k) = e(k)$$

若PID控制器采用采用增量式数字PID控制算法，则有

$$\left. \begin{aligned} \frac{\partial u(k)}{\partial O_1^{(3)}} &= e(k) - e(k-1) \\ \frac{\partial u(k)}{\partial O_2^{(3)}} &= e(k) \\ \frac{\partial u(k)}{\partial O_3^{(3)}} &= e(k) - 2e(k-1) + e(k-2) \end{aligned} \right\}$$

三，实验原理



$$\frac{\partial E(k)}{\partial \omega_{jl}^{(3)}} = \frac{\partial E(k)}{\partial y(k+1)} * \frac{\partial y(k+1)}{\partial u(k)} * \frac{\partial u(k)}{\partial O_l^{(3)}} * \frac{\partial O_l^{(3)}}{\partial net_l^{(3)}} * \frac{\partial net_l^{(3)}}{\partial \omega_{jl}^{(3)}}$$

其中： $E(k)$ 为指标函数 $\omega_{jl}^{(3)}$ 为隐含层-输出层权系数矩阵元素 $y(k+1)$ 为被控对象输出 $u(k)$ 为PID控制器输出 $O_l^{(3)}$ 为输出层输出 $net_l^{(3)}$ 为输出层输入；

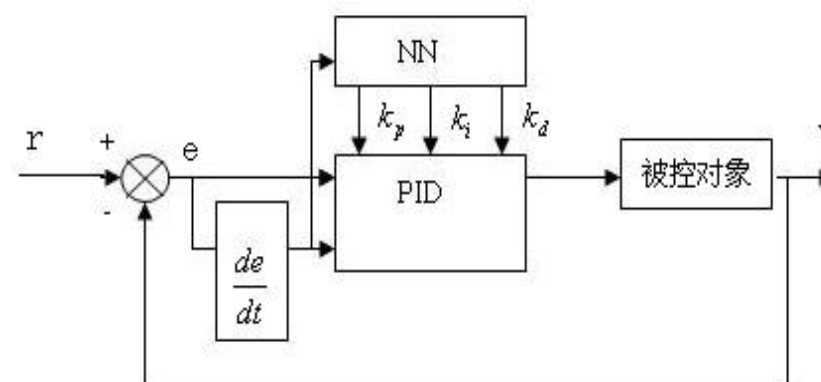
根据所选用神经网络的数学模型，易知：

$$\left. \begin{aligned} \frac{\partial E(k)}{\partial y(k+1)} &= -(r(k+1) - y(k+1)) = -e(k+1) \\ \frac{\partial u(k)}{\partial O_1^{(3)}} &= e(k) - e(k-1) \\ \frac{\partial u(k)}{\partial O_2^{(3)}} &= e(k) \\ \frac{\partial u(k)}{\partial O_3^{(3)}} &= e(k) - 2e(k-1) + e(k-2) \\ \frac{\partial O_l^{(3)}}{\partial net_l^{(3)}} &= g[net_l^{(3)}] \\ \frac{\partial net_l^{(3)}}{\partial \omega_{jl}^{(3)}} &= O_j^{(2)}(k) \end{aligned} \right\}$$

三，实验原理

$$\frac{\partial E(k)}{\partial \omega_{jl}^{(3)}} = \frac{\partial E(k)}{\partial y(k+1)} * \frac{\partial y(k+1)}{\partial u(k)} * \frac{\partial u(k)}{\partial O_i^{(3)}} * \frac{\partial O_i^{(3)}}{\partial net_i^{(3)}} * \frac{\partial net_i^{(3)}}{\partial \omega_{jl}^{(3)}}$$

$$\frac{\partial y(k+1)}{\partial u(k)}$$



直接的数学表达不容易获得，但我们可以使用它的符号函数来近似，仍可以保证参数修正方向的正确性，而由此造成其模的误差只影响参数调整的速度，它可以通过调整学习速率来得以补偿。

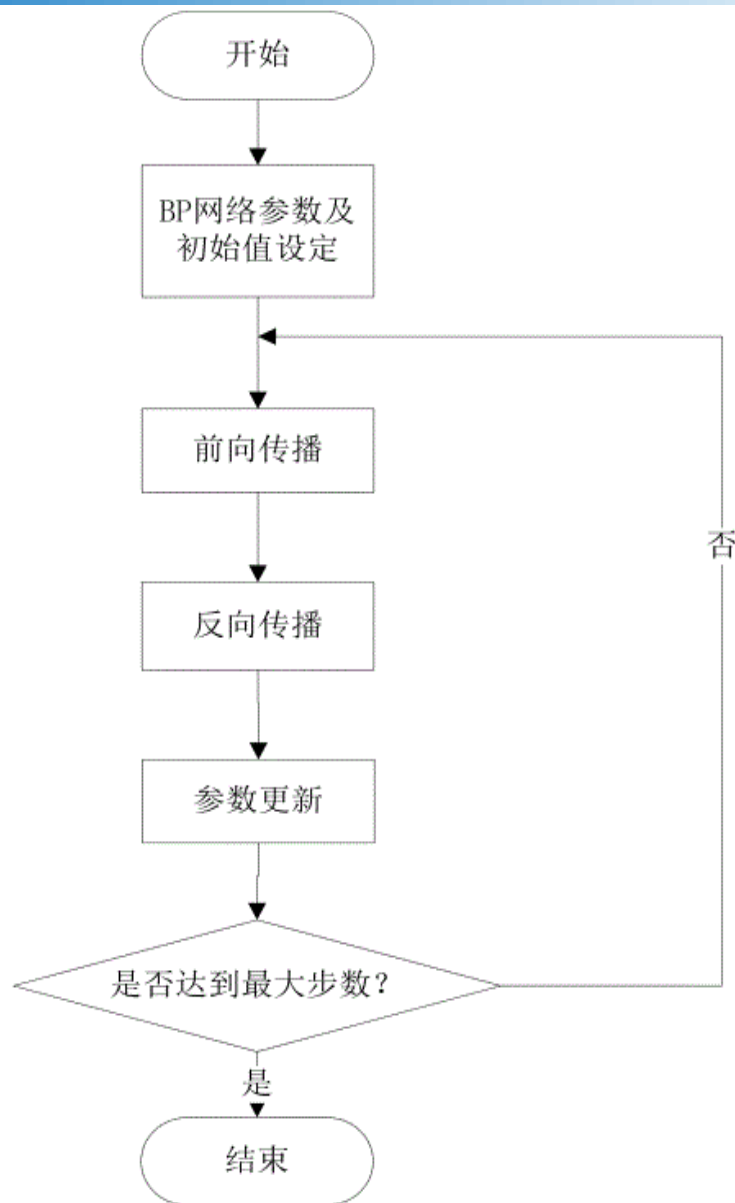
$$\frac{\partial E(k)}{\partial \omega_{jl}^{(3)}} = -e(k+1) * \operatorname{sgn}\left[\frac{\partial y(k+1)}{\partial u(k)}\right] * [e(k) - e(k-1)] * g[net_i^{(3)}] * O_j^{(2)}(k)$$

$$\left. \begin{aligned} \omega_{ij}^{(2)}(k+1) &= \omega_{ij}^{(2)}(k) - \eta * \frac{\partial E(k)}{\partial \omega_{ij}^{(2)}(k)} + \alpha [\omega_{ij}^{(2)}(k) - \omega_{ij}^{(2)}(k-1)] \\ \omega_{jl}^{(3)}(k+1) &= \omega_{jl}^{(3)}(k) - \eta * \frac{\partial E(k)}{\partial \omega_{jl}^{(3)}(k)} + \alpha [\omega_{jl}^{(3)}(k) - \omega_{jl}^{(3)}(k-1)] \end{aligned} \right\}$$

其中 η 代表了算法每次在梯度负方向搜索的步长，称为网络的学习速率，另外，如果考虑上次权值对本次权值变化的影响，需要加入动量（平滑）因子 α

三，实验原理

- 程序流程
- 步骤1：设定初始状态和参数初始值，包括随机产生初始BP神经网络权值系数，设定初始输入输出值为零，设定学习速率和惯性系数，计数器设为 $k=1$ ，并设定计数上限等。
- 步骤2：计算产生BP神经网络隐含层输入。本程序为采样获得 $e(k)$ ，并结合储存的 $e(k-1)$ ， $e(k-2)$ ，及常数1作为隐含层输入。前两次的 $e(k-1)$ ， $e(k-2)$ 并未真实产生直接取0。
- 步骤3：前向传播计算。包括：（1）BP神经网络前向传播计算，得到输出层输出，（2）增量式PID控制器计算控制器输出；（3）被控对象模型计算输出值
- 步骤4：反向传播计算。包括：（1）修正输出层的权系数；（2）修正隐含层的权系数；
- 步骤5：参数更新
- 步骤6：如果 k 达到设定的次数上限，则结束；否则， $k=k+1$ ，并返回步骤2。



四，代码理解



- 学习率
- 冲量
- 输入输出维度
- 隐含层
- 初始权重

```
cankao.m  BPPID.m  +
1 - clear all;
2 - close all;
3 - xite=0.65;
4 - alfa=0.01;
5 - IN=4;
6 - H=10;
7 - Out=3;
8 - wi=[ 0.4634 -0.4173  0.3190  0.0
9         0.1839  0.3021  0.1112  0.0
10        -0.3182  0.0470  0.0850 -0.00
11        -0.6266  0.0846  0.3751 -0.6900
12        -0.3224  0.1440 -0.2873 -0.0193
13        -0.0232 -0.0992  0.2636  0.2011
14        -0.4502 -0.2928  0.0062 -0.5640
15        -0.1975 -0.1332  0.1981  0.0422
16         0.0521  0.0673 -0.5546 -0.4830
17        -0.6016 -0.4097  0.0338 -0.1503
18      ];
19 - wi_1=wi;wi_2=wi;wi_3=wi;
20
21 - wo=[-0.1620  0.3674  0.1959
22        -0.0337 -0.1563 -0.1454
23         0.0898  0.7239  0.7605
24         0.3349  0.7683  0.4714
25         0.0215  0.5896  0.7143
26        -0.0914  0.4666  0.0771
27         0.4270  0.2436  0.7026
28         0.0215  0.4400  0.1121
29         0.2566  0.2486  0.4857
30         0.0198  0.4970  0.6450
31      ];
32 - wo_1=wo;wo_2=wo;wo_3=wo;
33
```

四，代码理解

- 相关参数初始化
- 被控对象实现
- 计算误差和对应PID分量的误差
- 神经网络的计算过程
- 获得Kp, Ki, Kd
- 计算控制器输出

```
error_2=0;
error_1=0;

ts=0.01;
for k=1:1:600
    time(k)=k*ts;
    rin(k)=1;
    %rin(k)=sin(2*3.1415926*time(k));
    a(k)=1.2*(1-0.8*exp(-0.1*k));
    yout(k)=a(k)*y_1/(1+y_1^2)+u_1;
    error(k)=rin(k)-yout(k);
    xi=[rin(k),yout(k),error(k),1];
    x(1)=error(k)-error_1;
    x(2)=error(k);
    x(3)=error(k)-2*error_1+error_2;
    epid=[x(1);x(2);x(3)];

    I=xi*wi';%Middle Layer
    for j=1:1:H
        Oh(j)=(exp(I(j))-exp(-I(j)))/(exp(I(j))+exp(-I(j)));
    end

    K=wo'*Oh;%Output Layer
    for l=1:1:Out
        K(l)=exp(K(l))/(exp(K(l))+exp(-K(l)));
    end

    kp(k)=K(1);ki(k)=K(2);kd(k)=K(3);%Getting kp,ki,kd
    Kpid=[kp(k),ki(k),kd(k)];

    du(k)=Kpid*epid;
    u(k)=u_1+du(k);
    if u(k)>=10 % Restricting the output of controller
        u(k)=10;
    end
    if u(k)<=-10
        u(k)=-10;
    end
end
```

四，代码理解



- 开始反向传播误差
- 先 W_o
- 再 W_i
- 为下一轮做准备
- 然后画图

```
dyu(k)=sign((yout(k)-y_1)/(u(k)-u_1+0.0000001));

%Output layer
] for j=1:1:Out
    dK(j)=2/(exp(K(j))+exp(-K(j)))^2;
- end

] for l=1:1:Out
    delta3(l)=error(k)*dyu(k)*epid(l)*dK(l);
- end

] for l=1:1:Out
    for i=1:1:H
        d_wo=xite*delta3(l)*Oh(i)+alfa*(wo_1-wo_2);
    - end
- end
wo=wo_1+d_wo+alfa*(wo_1-wo_2);

%Hidden layer
] for i=1:1:H
    dO(i)=4/(exp(I(i))+exp(-I(i)))^2;
- end

segma=delta3*wo';
] for i=1:1:H
    delta2(i)=dO(i)*segma(i);
- end

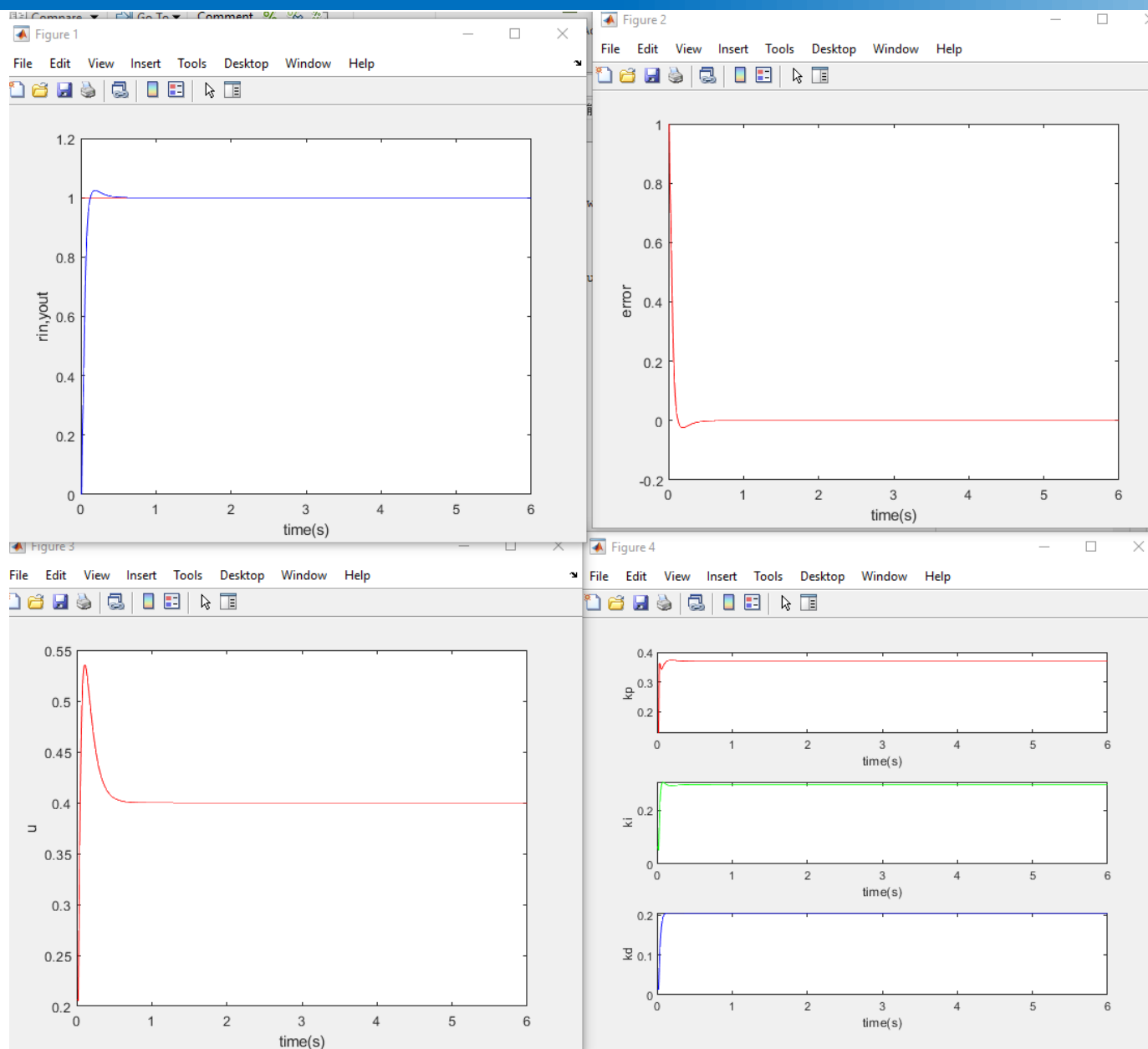
d_wi=xite*delta2'*xi+alfa*(wi_1-wi_2);
wi=wi_1+d_wi;
```

```
d_wi=xite*delta2'*xi+alfa*(wi_1-wi_2);
wi=wi_1+d_wi;

%Parameters Update
u_5=u_4;u_4=u_3;u_3=u_2;u_2=u_1;u_1=u(k);
y_2=y_1;y_1=yout(k);
wo_3=wo_2;
wo_2=wo_1;
wo_1=wo;
wi_3=wi_2;
wi_2=wi_1;
wi_1=wi;
error_2=error_1;
error_1=error(k);
end

%c
figure(1);
plot(time,rin,'r',time,yout,'b');
xlabel('time(s)');ylabel('rin,yout');
figure(2);
plot(time,error,'r');
xlabel('time(s)');ylabel('error');
figure(3);
plot(time,u,'r');
xlabel('time(s)');ylabel('u');
figure(4);
subplot(311);
plot(time,kp,'r');
xlabel('time(s)');ylabel('kp');
subplot(312);
plot(time,ki,'g');
xlabel('time(s)');ylabel('ki');
subplot(313);
plot(time,kd,'b');
xlabel('time(s)');ylabel('kd');
```


五, 输出结果



三、任务与报告



- 任务一：BP网络的学习速度和冲量的影响
- 任务二：尝试初始权值全是0, 0.5, -0.5, 随机
- 任务三：尝试H个数, 5,20,50,100,
- 任务四：当跟踪曲线是 $\%rin(k)=\sin(2*3.1415926*time(k))$, 是否可以收敛
- 学有余力的同学：两层隐含层

三、任务与报告



- 实验时间：5 – 8节
- 报告提交时间：下周三之前
- 报告内容：
 - 简述任务一，任务二，和任务三实现过程，分别计算其最优超调量，调节时间，与稳态误差
 - 简述任务四实现过程，计算其最大超调量，调节时间，与稳态误差
 - 学有余力的同学请描述实现过程，和结果曲线